DOCUMENT

20

Code Reference

Every module, class and function, generated from the docstrings

Every module, class and function, with its purpose, arguments, return value and errors. **Generated** from the docstrings and doc comments in the source by `scripts/generate_code_reference.py`; do not edit it by hand. Private helpers (names starting with an underscore) and individual tests are left out.

Domain layer

Business rules. No web or storage code.

`backend/src/invoice_app/domain/__init__.py`

Business rules. No web or storage code lives here.

`backend/src/invoice_app/domain/approval.py`

Value objects for approvals: who is approving, and what has been approved.

class `Approver`

Who is trying to approve: a username and the role they signed in with.

class `ApprovalRecord`

One recorded approval on an invoice. Invoices above the Director limit collect two.

`backend/src/invoice_app/domain/approval_policy.py`

Approval limits by role. Pure rules: no storage, no clock, no mutation.

- A Manager may approve invoices up to 50,000 (inclusive).
- A Director may approve invoices up to 500,000 (inclusive).
- Above 500,000 two distinct approvers are needed and the second must be a Director.
- The same user never counts as both approvers (usernames compared trimmed and case-insensitively).

Design (SOLID):

- Each rule is a small class with one reason to change (single responsibility).
- A policy is an ordered list of rules. A new rule is a new class added to the list; no existing rule or the policy itself is edited (open/closed).
- Rules and policy share narrow contracts (`ApprovalRule`), so any rule can stand in for another (Liskov).
- Limits are data (`ApprovalLimits`), passed in rather than read from globals (dependency inversion).

class `ApprovalLimits`

The two amounts the rules turn on. Inclusive: an invoice equal to a limit is within it.

class `ApprovalAttempt`

Everything a rule may look at: the invoice amount, who is approving, and who already approved.

class `ApprovalRule` (Protocol)

One reason an approval may be refused. Returns the reason, or None when this rule has no objection.

`refusal`

```
def refusal(self, attempt: ApprovalAttempt) -> Optional[str]
```

Return the reason this rule refuses the attempt, or `None` when it has no objection.

class `ApproverRoleRule`

Only the listed roles may approve anything at all.

`__init__`

```
def __init__(self, allowed_roles: Sequence[str]=APPROVER_ROLES) -> None
```

Args

allowed_roles: Roles that may approve at all. Defaults to Manager and Director.

`refusal`

```
def refusal(self, attempt: ApprovalAttempt) -> Optional[str]
```

Refuse any approver whose role is not in the allowed list (for example Admin).

class `ManagerLimitRule`

Above the Manager limit (and up to the Director limit) a Director must approve.

`__init__`

```
def __init__(self, limits: ApprovalLimits) -> None
```

Args

limits: The Manager and Director limits.

`refusal`

```
def refusal(self, attempt: ApprovalAttempt) -> Optional[str]
```

Refuse a non-Director when the amount is above the Manager limit and up to the Director limit.

Above the Director limit this rule is silent: the two-approver rules decide there.

class `DistinctApproversRule`

Above the Director limit the same user may not approve twice.

`__init__`

```
def __init__(self, limits: ApprovalLimits) -> None
```

Args

limits: The Manager and Director limits.

`refusal`

```
def refusal(self, attempt: ApprovalAttempt) -> Optional[str]
```

Above the Director limit, refuse an approver who has already approved this invoice.

class `SecondApproverMustBeDirectorRule`

Above the Director limit the second approver must be a Director.

`__init__`

```
def __init__(self, limits: ApprovalLimits) -> None
```

Args

limits: The Manager and Director limits.

`refusal`

```
def refusal(self, attempt: ApprovalAttempt) -> Optional[str]
```

Above the Director limit, refuse a second approver who is not a Director.

The first approver may be a Manager or a Director; only a second approval is constrained.

class `ApprovalPolicy`

Decides whether an approval is allowed by asking each rule in order; the first objection wins.

`__init__`

```
def __init__(self, rules: Sequence[ApprovalRule], limits: ApprovalLimits=ApprovalLimits()) ->
None
```

Args

rules: Rules in the order their reasons should be reported; the first objection wins. limits: Used for how many approvals an invoice needs.

`refusal_reason`

```
def refusal_reason(self, amount: Decimal, approver: Approver, prior_approvals:
Sequence[ApprovalRecord]) -> Optional[str]
```

Ask each rule in order and return the first objection.

Args

- amount: The invoice amount.
- approver: Who is trying to approve.
- prior_approvals: Approvals already recorded on the invoice.

Returns

- The reason the approval is refused, or `None` when it is allowed.

`required_approvals`

```
def required_approvals(self, amount: Decimal) -> int
```

How many distinct approvals the invoice needs before it counts as Approved.

`standard_policy`

```
def standard_policy(limits: ApprovalLimits=ApprovalLimits()) -> ApprovalPolicy
```

The rules of this feature, in the order their reasons should be reported.

`required_approvals`

```
def required_approvals(amount: Decimal) -> int
```

How many distinct approvals an invoice of this amount needs: 1, or 2 above the Director limit.

`rejection_reason`

```
def rejection_reason(amount: Decimal, approver: Approver, prior_approvals:
Sequence[ApprovalRecord]) -> Optional[str]
```

Why this approver may not approve now, or None when the approval is allowed.

`is_same_user`

```
def is_same_user(first_username: str, second_username: str) -> bool
```

True when two usernames are the same person: equal after trimming spaces and ignoring case.

`backend/src/invoice_app/domain/approval_service.py`

The approval workflow: submit, approve and reject an invoice.

class `ApprovalDecisions` (Protocol)

What the service needs to know from the rules: may this approval happen, and how many are required.

`refusal_reason`

```
def refusal_reason(self, amount: Decimal, approver: Approver, prior_approvals:
Sequence[ApprovalRecord]) -> Optional[str]
```

Return why this approval is not allowed, or `None` when it is.

`required_approvals`

```
def required_approvals(self, amount: Decimal) -> int
```

Return how many distinct approvals an invoice of this amount needs.

class `ApprovalService`

Moves an invoice through its lifecycle: Draft -> PendingApproval -> Approved or Rejected.

The service coordinates; it does not own the limits. Who may approve is decided by the injected policy, and time comes from the injected clock, so both can be replaced without touching this class.

`__init__`

```
def __init__(self, store: ApprovalStore, policy: Optional[ApprovalDecisions]=None, clock:
Callable[[], datetime]=utc_now) -> None
```

Args

store: Finds and saves invoices (only those two operations are needed). policy: Decides who may approve and how many approvals are required. Defaults to the standard rules. clock: Returns the current time; replace it in tests to control timestamps.

`submit`

```
def submit(self, invoice_id: str) -> Invoice
```

Move a Draft invoice to PendingApproval.

Raises

- InvoiceNotFoundError: unknown id.
- InvalidStateError: the invoice is not a Draft.

`approve`

```
def approve(self, invoice_id: str, approver: Approver) -> Invoice
```

Record one approval; mark the invoice Approved when enough have been given.

All checks (state, then the policy) run before anything is changed, so a refused attempt leaves the invoice exactly as it was.

Raises

- InvoiceNotFoundError: unknown id.
- InvalidStateError: the invoice is not pending.
- ApprovalNotPermittedError: the policy refuses this approver; the message is the reason.

`reject`

```
def reject(self, invoice_id: str, approver: str, reason: str) -> Invoice
```

Reject a pending invoice. A reason is required.

Raises

- DomainValidationError: the reason is blank.
- InvoiceNotFoundError: unknown id.
- InvalidStateError: the invoice is not pending.

`required_approvals`

```
def required_approvals(self, amount: Decimal) -> int
```

Exposed so the API can show "N of M approvals" using the same policy the service enforces.

`backend/src/invoice_app/domain/discount_calculator.py`

Discount arithmetic for pricing. Independent of approvals; kept from the original demo app.

class `DiscountCalculator`

Volume and loyalty discount rules used when pricing invoices.

`quantity_discount`

```
def quantity_discount(self, quantity: int) -> Decimal
```

Volume discount rate: 5% from 10 units, 10% from 50, 15% from 100.

`loyalty_discount`

```
def loyalty_discount(self, years_as_customer: int) -> Decimal
```

Loyalty discount rate: 1% per year as a customer, capped at 10%.

`combined_discount`

```
def combined_discount(self, quantity: int, years_as_customer: int, customer_type: Optional[str])
-> Decimal
```

Sum of the volume and loyalty rates, plus 5% for a `Nonprofit` customer.

`apply_discount`

```
def apply_discount(self, amount: Decimal, discount_rate: Decimal) -> Decimal
```

Apply a rate to an amount and round to the cent (banker's rounding).

`backend/src/invoice_app/domain/errors.py`

Domain errors, each described in business terms.

The API layer maps them to HTTP statuses in `api/error_handling.py`; the domain never mentions HTTP.

class `InvoiceNotFoundError` (LookupError)

The invoice id does not exist. Mapped to HTTP 404.

class `InvalidStateError` (Exception)

The invoice is in the wrong lifecycle state for the action. Mapped to HTTP 409.

class `DomainValidationError` (ValueError)

The caller supplied unusable input (for example a blank rejection reason). Mapped to HTTP 400.

class `ApprovalNotPermittedError` (Exception)

The approver may not approve this invoice. The message is the clear reason. Mapped to HTTP 403.

`backend/src/invoice_app/domain/invoice.py`

The Invoice entity.

`utc_now`

```
def utc_now() -> datetime
```

Return the current time as a timezone-aware UTC datetime.

This is the domain's default clock. Services accept another clock so tests can control time.

class `Invoice`

An invoice moving through approval.

Attributes

- id: Invoice number such as `INV-1001`; assigned by the repository when empty.
- customer_name: Who is being billed.
- amount: Money as `Decimal` (never `float`) so limit comparisons are exact.

- status: Current lifecycle state.
- created_at: When the invoice was created (UTC).
- decided_by: Username of the person whose action made the final decision, if decided.
- decided_at: When the final decision was made (UTC).
- decision_note: The reason, for a rejection.
- approvals: Accepted approvals so far, oldest first. Above the Director limit two are needed.

`backend/src/invoice_app/domain/invoice_status.py`

The lifecycle states of an invoice.

class `InvoiceStatus` (str, Enum)

Where an invoice is in its life: Draft, then PendingApproval, then Approved or Rejected.

The string values are what the API returns and the UI shows.

`backend/src/invoice_app/domain/repository.py`

Storage contracts, split by who needs what (interface segregation).

A consumer depends on the smallest contract that covers its needs: the approval service needs `ApprovalStore` (find one invoice, save it), read-only endpoints need `InvoiceReader`, create and delete need `InvoiceWriter`. `InvoiceRepository` is the full contract an implementation offers.

class `InvoiceReader` (Protocol)

Read-only access to invoices.

`get_all`

```
def get_all(self) -> list[Invoice]
```

Return every invoice. The list is a snapshot: changing it does not change the store.

`get_by_id`

```
def get_by_id(self, invoice_id: str) -> Optional[Invoice]
```

Return the invoice with this id, or `None` when there is none (not an error).

`get_by_ids`

```
def get_by_ids(self, invoice_ids: Iterable[str]) -> list[Invoice]
```

Return the invoices that exist among these ids; unknown ids are ignored.

class `InvoiceWriter` (Protocol)

Operations that change what is stored.

`add`

```
def add(self, invoice: Invoice) -> Invoice
```

Store a new invoice and return it. An invoice without an id is given one.

`update`

```
def update(self, invoice: Invoice) -> None
```

Replace the stored invoice that has the same id.

Raises

- InvoiceNotFoundError: if no such invoice exists (never a silent no-op).

`remove`

```
def remove(self, invoice_id: str) -> bool
```

Delete the invoice; return `True` if something was removed, `False` if the id was unknown.

class `ApprovalStore` (Protocol)

All the approval service needs from storage.

`get_by_id`

```
def get_by_id(self, invoice_id: str) -> Optional[Invoice]
```

Return the invoice with this id, or `None`.

`update`

```
def update(self, invoice: Invoice) -> None
```

Save the invoice. Raises `InvoiceNotFoundError` when it does not exist.

class `InvoiceRepository` (InvoiceReader, InvoiceWriter, Protocol)

The full repository contract implemented by the in-memory store (and by test fakes).

API layer

HTTP routers, sign-in, serialization and the in-memory repository.

`backend/src/invoice_app/api/__init__.py`

The HTTP layer: routers, sign-in, serialization and the in-memory repository. Translates between the web and the domain; contains no business rules.

`backend/src/invoice_app/api/auth/__init__.py`

Sign-in and sessions: who the caller is, and which roles may call which endpoint.

`backend/src/invoice_app/api/auth/demo_users.py`

The demo accounts. Local, in-memory and not real credentials.

class `DemoUser`

A demo account: username, password and the role it signs in with.

`backend/src/invoice_app/api/auth/dependencies.py`

FastAPI dependencies for reading the caller's token and session, and for restricting endpoints by role.

`bearer_token`

```
def bearer_token(request: Request) -> str | None
```

Extract the token from an `Authorization: Bearer <token>` header, or `None` if absent.

`current_session`

```
def current_session(request: Request) -> UserSession
```

Return the session the middleware attached to this request.

`require_role`

```
def require_role(*roles: str)
```

Allows the endpoint only for the listed roles; everyone else gets 403.

`backend/src/invoice_app/api/auth/middleware.py`

The session check applied to every API call.

`install_session_middleware`

```
def install_session_middleware(app: FastAPI) -> None
```

Every /api route except login needs a valid session; the session is attached to the request.

`backend/src/invoice_app/api/auth/session_store.py`

In-memory sign-in sessions keyed by an opaque bearer token. Tokens do not expire (known limit, review finding F11).

class `UserSession`

Who is signed in: a username and the role they signed in with.

class `SessionStore`

Keeps signed-in sessions in memory, keyed by an opaque bearer token.

`__init__`

```
def __init__(self) -> None
```

Create an empty, thread-safe store.

`create`

```
def create(self, session: UserSession) -> str
```

Store the session and return a new random token for it.

`get`

```
def get(self, token: Optional[str]) -> Optional[UserSession]
```

Return the session for a token, or `None` for a missing or unknown token.

`remove`

```
def remove(self, token: str) -> None
```

Forget a token (sign out). Unknown tokens are ignored.

`backend/src/invoice_app/api/auth/user_directory.py`

Where user accounts come from, behind a small contract so the login endpoint does not depend on how they are stored.

class `AuthenticatedUser`

A user who passed authentication: username and role.

class `UserDirectory` (Protocol)

Who may sign in. The login endpoint depends on this, not on where the accounts are kept.

`authenticate`

```
def authenticate(self, username: str, password: str) -> Optional[AuthenticatedUser]
```

Return the user when the username and password match, otherwise `None`.

class `DemoUserDirectory`

The fixed demo accounts. A real deployment would provide another UserDirectory without touching the router.

`__init__`

```
def __init__(self, users: tuple[DemoUser, ...]=DEMO_USERS) -> None
```

Args

users: The accounts to accept. Defaults to the fixed demo accounts.

`authenticate`

```
def authenticate(self, username: str, password: str) -> Optional[AuthenticatedUser]
```

Return the matching demo user, or `None` for a wrong username or password.

`backend/src/invoice_app/api/error_handling.py`

How failures become HTTP responses. This is the only place that knows both domain errors and status codes.

`describe_validation_error`

```
def describe_validation_error(error: RequestValidationError) -> str
```

One readable sentence for the first invalid field; never echoes internals back to the client.

`register_error_handlers`

```
def register_error_handlers(app: FastAPI) -> None
```

Install the handlers that turn domain errors, HTTP errors and invalid requests into `{"error": ...}` responses.

`backend/src/invoice_app/api/in_memory_repository.py`

The in-memory invoice store used by the demo app: no database, resets on restart.

class `InMemoryInvoiceRepository`

In-memory store so the demo app runs with no database. Seeded with a few invoices.

`__init__`

```
def __init__(self) -> None
```

Create the store seeded with demo invoices, including INV-2001 (75,000.00) and INV-2002 (600,000.00) for trying the approval limits.

`get_all`

```
def get_all(self) -> list[Invoice]
```

Return a snapshot list of every invoice.

`get_by_id`

```
def get_by_id(self, invoice_id: str) -> Optional[Invoice]
```

Return the invoice with this id, or `None`.

`get_by_ids`

```
def get_by_ids(self, invoice_ids: Iterable[str]) -> list[Invoice]
```

Return the invoices whose ids are in the given collection; unknown ids are ignored.

`add`

```
def add(self, invoice: Invoice) -> Invoice
```

Store the invoice, giving it the next `INV-<n>` id when it has none.

`update`

```
def update(self, invoice: Invoice) -> None
```

Replace the stored invoice with the same id; raise `InvoiceNotFoundError` if there is none.

`remove`

```
def remove(self, invoice_id: str) -> bool
```

Delete the invoice; return whether anything was removed.

`backend/src/invoice_app/api/main.py`

Composition root: the one place that chooses concrete implementations and wires them together.

`create_app`

```
def create_app(repository: Optional[InvoiceRepository]=None, users: Optional[UserDirectory]=None,
policy: Optional[ApprovalDecisions]=None) -> FastAPI
```

Builds the app. Every collaborator can be replaced by passing another implementation; defaults suit the demo.

`backend/src/invoice_app/api/routers/__init__.py`

One FastAPI router per resource (auth, invoices, approvals). Routers translate HTTP to domain calls and back.

`backend/src/invoice_app/api/routers/approvals.py`

Approval endpoints. The router lets Manager, Director and Admin in; the domain decides who may actually approve.

class `RejectRequest` (BaseModel)

Body of the reject call: the reason (required by the domain, so an empty body gives 400).

`approve`

```
def approve(invoice_id: str, session: UserSession=Depends(current_session), approvals:
ApprovalService=Depends(get_approvals))
```

Record the signed-in user's approval. 403 with the reason when the rules refuse it; the invoice is then unchanged.

`reject`

```
def reject(invoice_id: str, body: RejectRequest, session: UserSession=Depends(current_session),
approvals: ApprovalService=Depends(get_approvals))
```

Reject a pending invoice with a reason.

`backend/src/invoice_app/api/routers/auth.py`

Sign-in and sign-out endpoints.

class `LoginRequest` (BaseModel)

Body of `POST /api/auth/login`.

`login`

```
def login(body: LoginRequest, request: Request)
```

Exchange a username and password for a bearer token (401 for a wrong pair).

`logout`

```
def logout(request: Request)
```

Forget the caller's token. Always 204.

`backend/src/invoice_app/api/routers/invoices.py`

Invoice endpoints: list, get, create, submit and (Admin only) delete.

class `CreateInvoiceRequest` (BaseModel)

Body of `POST /api/invoices`. The amount must be positive with at most 2 decimals and 12 digits before the point.

`get_repository`

```
def get_repository(request: Request) -> InvoiceRepository
```

Dependency: the app's repository.

`get_approvals`

```
def get_approvals(request: Request) -> ApprovalService
```

Dependency: the app's approval service.

`present`

```
def present(invoice: Invoice, approvals: ApprovalService) -> dict
```

The invoice as the UI sees it, with the approval count taken from the policy that is actually enforced.

`get_all`

```
def get_all(status: Optional[str]=None, reader: InvoiceReader=Depends(get_repository), approvals:
ApprovalService=Depends(get_approvals))
```

List invoices, optionally filtered by `?status=` (case-insensitive; 400 for an unknown status).

`get_by_id`

```
def get_by_id(invoice_id: str, reader: InvoiceReader=Depends(get_repository), approvals:
ApprovalService=Depends(get_approvals))
```

Return one invoice, or 404 with an empty body.

`create`

```
def create(body: CreateInvoiceRequest, writer: InvoiceWriter=Depends(get_repository), approvals: ApprovalService=Depends(get_approvals))
```

Create a Draft invoice (400 for a blank customer or an unusable amount).

`submit`

```
def submit(invoice_id: str, approvals: ApprovalService=Depends(get_approvals))
```

Send a Draft invoice for approval.

`delete`

```
def delete(invoice_id: str, writer: InvoiceWriter=Depends(get_repository))
```

Delete an invoice. Admin only; 404 when it does not exist.

`backend/src/invoice_app/api/serialization.py`

Turns an Invoice into the JSON shape the web UI consumes.

`invoice_to_json`

```
def invoice_to_json(invoice: Invoice, required_approvals: int) -> dict
```

Same camelCase shape the web UI has always consumed.

`backend/src/invoice_app/api/static_ui.py`

Serves the web UI files so a browser always asks again instead of showing an old copy.

class `RevalidatedStaticFiles` (StaticFiles)

StaticFiles that tells browsers to re-check every file (`Cache-Control: no-cache`).

Without it a browser may keep using an old `styles.css` or `app.js` after they change. The files are small and ETags still let the browser skip the download when nothing changed.

`get_response`

```
async def get_response(self, path: str, scope: Scope) -> Response
```

Return the normal response with the no-cache header added.

Hooks

Hooks that log every AI interaction and check outputs.

`.claude/hooks/check_summary.py`

PostToolUse hook for Bash: turn test/mutation/e2e output into ONE identically formatted line.

```
CHECK <name> | <metric>=<value> ... | result=<pass|fail>
```

The line is handed back to the agent as additionalContext and also appended to the AI log, so every run of pytest, mutmut or Playwright is reported in the same shape no matter who ran it.

`first_int`

```
def first_int(pattern: str, text: str) -> int
```

The first captured integer in the text, or 0 when the pattern does not match.

`summarize`

```
def summarize(command: str, output: str) -> str | None
```

Turn pytest, mutmut or Playwright output into one `CHECK <name> | metrics | result=pass|fail` line, or `None` for other commands.

`main`

```
def main() -> None
```

After a Bash or PowerShell call, log the check summary and hand it back to the agent as extra context.

`.claude/hooks/log_event.py`

Append one structured line per AI interaction to logs/ai-interactions.jsonl.

Wired to UserPromptSubmit, PostToolUse, SubagentStop and Stop in .claude/settings.json. Every line has the same keys, in the same order (see `FIELDS`), covering: the AI tool and model, the agent and its mode, the request, skills / sub-agents / MCP tools / tools used, what the AI suggested, the human decision, how the action was verified, limitations, status and the files touched. Empty facts are `null` or `[]`, never missing.

Traceability: a prompt record gets an `id`; every record that follows (tool calls, sub-agent reports, checks) lists it in `related_ids` and `parent_task`, so each action can be traced to the request that caused it.

Stdlib only. Never fails the session: any error is swallowed and the hook exits 0.

Manual use (to record work done outside a hook): log_event.py --manual EVENT "summary" [agent] [status] log_event.py --human "decision or correction" ["what it was about"]

`project_dir`

```
def project_dir() -> Path
```

The project folder: `CLAUDE_PROJECT_DIR` when Claude Code sets it, otherwise two levels above this file.

`local_path_forms`

```
def local_path_forms() -> list[tuple[str, str]]
```

(path text, placeholder) pairs, longest first, so the log never carries the user name or machine paths.

`clean`

```
def clean(text: object, limit: int=MAX_SUMMARY) -> str
```

Make text safe for the log: collapse whitespace, replace local paths and the account name, redact secrets, cap the length at `limit` characters.

`relative_path`

```
def relative_path(path_text: object) -> str
```

A file path relative to the project, with forward slashes (or the cleaned text when it is outside the project).

`load_defaults`

```
def load_defaults() -> dict
```

Tool, model and context from `.claude/log-defaults.json`; hooks cannot see the model, so the file names it.

`now_utc`

```
def now_utc() -> datetime
```

The current time in UTC, to the second.

`make_id`

```
def make_id(moment: datetime, agent: str, event: str, salt: str) -> str
```

A readable unique id: `20261019T063542Z-main-userpromptsubmit-3fa9c1`.

`new_record`

```
def new_record(**fields) -> dict
```

Build a full record: the defaults for every key in `FIELDS`, overridden by `fields`.

Args

- **fields: any of the keys in `FIELDS`. `event` is required; `timestamp` defaults to now.

Returns

- A dict with exactly the keys in `FIELDS`, in that order.

`describe_tool`

```
def describe_tool(name: str, tool_input: dict) -> str
```

One short line describing a tool call (the file for Edit/Read, the command for Bash, and so on).

`tool_outcome`

```
def tool_outcome(response: object) -> str
```

`ok` or `error` from a tool's response.

`infer_stage`

```
def infer_stage(tool: str, tool_input: dict) -> str
```

Which lifecycle stage a tool call belongs to, from the file it touches or the command it runs.

`agent_kind`

```
def agent_kind(name: str) -> str
```

`custom` when the sub-agent is defined in `.claude/agents`, otherwise `built-in`.

`skill_kind`

```
def skill_kind(name: str) -> str
```

`custom` when the skill lives in `.claude/skills`, otherwise `built-in`.

`capabilities_used`

```
def capabilities_used(tool: str, tool_input: dict, outcome: str) -> dict
```

The skills, sub-agents and MCP tools a tool call used, in the log's nested shapes.

`state_file`

```
def state_file() -> Path
```

Where the latest human prompt is remembered, so later records can point back to it.

`last_prompt`

```
def last_prompt() -> dict
```

`{"id", "request"}` of the latest human prompt, or empty when none is remembered.

`remember_prompt`

```
def remember_prompt(record_id: str, request: str) -> None
```

Remember the latest human prompt for the records that follow.

`is_system_message`

```
def is_system_message(prompt: str) -> bool
```

True for text the harness injects as a "prompt" (a sub-agent hand-back, a task notification), not human input.

`agent_fields`

```
def agent_fields(data: dict) -> dict
```

The agent identity fields: the main Claude Code agent, or a sub-agent when the hook payload names one.

`build_record`

```
def build_record(data: dict) -> dict
```

Turn a hook payload into one full log record (see `FIELDS`).

`append`

```
def append(record: dict) -> None
```

Append one record to `logs/ai-interactions.jsonl` with a single binary write so parallel hooks never interleave lines.

`main`

```
def main() -> None
```

Read the hook payload from stdin and log it; or, with `--manual` / `--human`, log what the command line says.

`.claude/hooks/test_log_event.py`

Tests for the AI-interaction logging hook (`log_event.py`): one fixed record shape, traceable links. Run with `python -m pytest .claude/hooks -q`.

`hook`

```
def hook(tmp_path, monkeypatch)
```

The hook module, pointed at an empty temporary project so tests never touch the real log.

`.claude/hooks/validate_review.py`

SubagentStop hook: the pr-reviewer agent must answer in the one structured findings format.

Accepted answer: a ```json fenced array (possibly empty) of objects with exactly the keys id (F1, F2, ...), severity (high|medium|low), location (path:line), issue, suggestion If the answer does not parse or breaks the schema, the subagent is blocked from stopping and told why, so the review always comes back in the same shape. Skills/agents: see .claude/skills/review-findings.

`extract_json`

```
def extract_json(message: str)
```

Return the parsed content of the first `` `json block, or `` None`` when there is none.

`problems`

```
def problems(message: str) -> list[str]
```

Everything wrong with a reviewer's answer against the findings schema; an empty list means it is valid.

`main`

```
def main() -> None
```

Block the pr-reviewer agent from finishing until its answer follows the findings format.

Tests

What each test module covers, and the shared fake.

`backend/tests/fake_invoice_repository.py`

A test double for the invoice repository that behaves like the real one.

`backend/tests/test_api.py`

HTTP-level tests: status codes, messages, sign-in, validation and the approval flow through the real routers.

`backend/tests/test_approval_policy.py`

Unit tests for the pure approval rules. Boundaries are tested on both sides, in both directions.

`backend/tests/test_approval_service.py`

Service-level tests: the approval workflow against a fake repository, including that a refused attempt changes nothing.

`backend/tests/test_repository_contract.py`

One contract, every repository. Anything that claims to be an InvoiceRepository must pass these tests, so the fake used in unit tests and the in-memory store used by the app are interchangeable (Liskov).

Scripts

Helper scripts for cases, documentation and the demo video.

`scripts/build_docs.py`

Build the project documentation as PDFs.

```
python scripts/build_docs.py
```

Sources are Markdown files under docs/guides plus the spec, PR review, assignment and agent files already in the repository. Output in docs/pdf/: one separate PDF per document (divider page + content). Add --kit to build the three personal video-kit PDFs into For-Me-Video-Kit/ instead. Add --combined to also build one merged Invoice-Approval-Complete-Documentation.pdf (cover, contents, every document). Needs: pip install markdown pypdf playwright && playwright install chromium

`scripts/generate_code_reference.py`

Generate docs/guides/20-code-reference.md from the docstrings and doc comments in the code.

```
python scripts/generate_code_reference.py
```

The reference is built from the source itself (Python docstrings, JSDoc blocks in the web UI and the end-to-end page objects), so it cannot drift from the code. Rebuild the PDFs afterwards with `python scripts/build_docs.py`.

`scripts/make_demo_video.py`

Record the demo video.

```
python scripts/make_demo_video.py            # full recording (about 10 minutes of real time)
DEMO_FAST=1 python scripts/make_demo_video.py  # smoke run: no waiting for the voice, video not
in sync
```

Every command shown in the terminal scenes really runs while it is being recorded (pytest, a deliberate one-character break that is restored afterwards, mutmut in WSL, Playwright, the log). The UI scene drives the real app on a fresh server. The voiceover is Windows' built-in speech engine; captions repeat the narration.

Output: docs/video/demo-narrated.mp4 and docs/video/demo-captions-only.mp4 Needs: pip install imageio-ffmpeg playwright markdown pypdf (and WSL with the mutation venv, see scripts/run_mutation.sh)

`scripts/summarize_log.py`

Check the AI interaction log and write a readable summary of it.

```
python scripts/summarize_log.py            # write logs/ai-interactions-summary.md
python scripts/summarize_log.py --check    # only validate the log; exit 1 when a line is wrong
```

The log (`logs/ai-interactions.jsonl`) is the machine-readable evidence. The summary is the same evidence for a human reader: who did what with which tool and model, what the human decided, and which checks proved the result.

`scripts/try_all_cases.py`

Try every approval case against a running server and print PASS / FAIL for each.

```
python scripts/try_all_cases.py                     # starts its own fresh server on port 5058
python scripts/try_all_cases.py --url http://localhost:5000   # use a server you already started
```

Each case creates its own invoice, so cases do not depend on each other. Exit code is 0 only if every case passes. Uses only the standard library plus the app itself.

Web UI

Plain JavaScript, no build step. Business rules live on the server; the UI shows what the API returns.

`backend/src/invoice_app/api/wwwroot/app.js`

Invoice Approvals web UI (plain JavaScript, no build step).

Signs a user in, lists invoices, and lets approvers approve or reject pending ones. All business rules live on the server: this file only shows what the API returns, including the reason when an approval is refused.

Everything is wrapped in a function so nothing leaks into the global scope.

```
function el(tag, attrs = {}, ...children)
```

Invoice Approvals web UI (plain JavaScript, no build step).

Signs a user in, lists invoices, and lets approvers approve or reject pending ones. All business rules live on the server: this file only shows what the API returns, including the reason when an approval is refused.

Everything is wrapped in a function so nothing leaks into the global scope. / (() => { // Display formatting only. The server keeps exact decimals; the UI just shows them as dollars. const money = new Intl.NumberFormat('en-US', { style: 'currency', currency: 'USD' }); // Shorthand: find an element by id. const $ = (id) => document.getElementById(id); // The signed-in user ({ token, username, role }). Kept in sessionStorage so a page refresh does not sign out. let session = JSON.parse(sessionStorage.getItem('session') || 'null');

/** Build a DOM element. Using this instead of innerHTML means server text is never parsed as HTML. @param {string} tag Element name, for example 'button'. @param {Object} attrs Attributes; the key 'class' sets the class name. @param {...(Node|string)} children Child nodes or text.

```
async function api(path, options = {})
```

Call the JSON API with the signed-in user's bearer token. A 401 means the session is gone, so sign out. Any other failure throws an Error carrying the server's own message (the `error` field), so callers can show a clear reason to the user. @param {string} path Path under /api, for example '/invoices'. @param {RequestInit} options Fetch options. @returns {Promise} The parsed JSON body, or null for 204.

```
function show(id, visible)
```

Show or hide an element by id.

```
function showError(id, message)
```

Put a message in an alert paragraph (role=alert) and reveal it; an empty message hides it.

```
function render()
```

Switch between the sign-in view and the app view depending on whether someone is signed in.

```
async function refresh()
```

Reload invoices from the server and redraw both the invoices table and the pending approvals list.

```
function statusCell(invoice)
```

The Status cell: a coloured pill, and for a rejected invoice the reason and who rejected it, so everyone who can see the invoice can see why it was turned down.

```
function renderInvoices(invoices)
```

Draw the invoices table. Draft invoices get a Submit button; the status text is split into words (PendingApproval to Pending Approval).

```
function renderPending(pending)
```

Draw the pending approvals list: an Approve button and a reason box with a Reject button for each invoice. Invoices that need more than one approval show progress such as '1 of 2 approvals'.

```
async function decide(path, payload)
```

Send an approve or reject request, then reload. If the rules refuse it (HTTP 403) the server's reason is shown under the list and nothing else changes, because the server leaves the invoice untouched.

```
function signOut()
```

Forget the session locally, tell the server to forget the token (best effort), and show the sign-in view.

End-to-end page objects

Playwright page objects. Elements are found by role or label only.

`e2e/pages/LoginPage.ts`

```
export class LoginPage
```

Page object for the sign-in screen. Everything is found by role or label, never by CSS or XPath, so the tests read like what a user sees.

```
async open()
```

Open the app and wait for the Sign in heading.

```
async signInAs(username: string, password = 'ChangeMe123!')
```

Fill the form and sign in, then wait until the header shows who is signed in. @param username One of the demo accounts, for example 'demo.manager'. @param password Defaults to the shared demo password.

`e2e/pages/ApprovalsPage.ts`

```
export class ApprovalsPage
```

Page object for the Approvals and Invoices tabs. Found by role and accessible name only.

```
async open()
```

Switch to the Approvals tab and wait for its heading.

```
pendingEntry(invoiceId: string): Locator
```

The list item for one pending invoice.

```
async approve(invoiceId: string)
```

Press the Approve button of one invoice (its accessible name includes the invoice id).

```
decisionError(): Locator
```

The alert that shows why an approval was refused.

```
async openInvoicesTab()
```

Switch to the Invoices tab.

```
invoiceRow(invoiceId: string): Locator
```

The table row for one invoice.